Notas de exposición: Open AI Cubes

Guía para los tres integrantes. Duración sugerida: unos 15 minutos de exposición más la demo y las preguntas.

Reparto sugerido

Integrante	Diapositivas	Tema
A	1 a 6	Problema, arquitectura, hosts y clientes, CNN, micro chunks
B	7 a 10	Paralelo vs. distribuido, protocolo OC/1, diagrama, correctitud y fallos
C	11 a 15 + demo	Matriz de 12 casos, escalabilidad, carga, demo en vivo, conclusiones

Guion por diapositiva

1. Portada. Nombres y una frase: "Implementamos el cubo de procesamiento de imágenes de Open AI Cubes: una CNN que se entrena y responde en un clúster con nodos en Java, Python y Go".

2. Qué construimos. Remarcar las restricciones: solo librería estándar, sockets e hilos, sin HTTP/WebSocket/middleware. Clientes y servidor en Java 8+, nodos en tres lenguajes.

3. Arquitectura. Tres tipos de programa. Los clientes y los nodos abren la conexión hacia el servidor: por eso se pueden agregar máquinas sin configurar nada en el servidor.

4. Hosts y clientes. Puerto 5000 para clientes, 6000 para nodos. El backend vive dentro del servidor como módulo aparte (separarlo agregaría un salto de red sin ganar nada).

5. La CNN. Dos convoluciones con pooling extraen características; el MLP clasifica. 82,3 % de acierto en 4 000 imágenes de prueba. Mismos pesos iniciales en todas las mediciones.

6. Micro chunks. Un micro chunk es un rango de filtros, neuronas o canales. Cada hilo escribe solo en su rango, por eso no hacen falta candados. Entre capas hay una barrera porque la siguiente capa necesita el resultado completo.

7. Qué se paraleliza y qué se distribuye. Distribuido: imágenes del lote (paralelismo de datos). Paralelo: micro chunks dentro de cada nodo. Concurrente: el servidor atiende a muchos clientes. La suma de gradientes no depende de cómo se reparta el lote, por eso el resultado distribuido es igual al local.

8. Protocolo. Cabecera de 12 bytes: magic "OC", versión, tipo, id de petición, longitud. El id empareja cada respuesta con su petición, así un socket puede tener varias peticiones en curso.

9. Diagrama. Recorrer los tres flujos: registro, consulta de chat, paso de entrenamiento.

10. Correctitud y fallos. Prueba cruzada: los tres lenguajes dan el mismo gradiente que Java. Si se cae un nodo, su parte se reasigna. Mostrar en la demo apagando un nodo si el profesor lo pide.

11. Matriz de 12 casos. Mismas 1 024 imágenes y mismos pesos iniciales en todos los casos. Python es unas 10 veces más lento, pero escala mejor en proporción. Go con 4 hilos es el más rápido en una máquina (1,9 s). Medido en una sola máquina de 4 núcleos: en el laboratorio se repite con los mismos scripts.

12. Escalabilidad. El speedup llega a ≈3× en 4 núcleos. No es lineal por la parte secuencial del servidor (sumar y actualizar pesos), la comunicación (≈2 MB por nodo y por lote) y porque aquí los nodos comparten CPU. El clúster mixto rinde menos porque se espera al nodo más lento en cada lote.

13. Carga. 0 errores en 31 200 consultas. El throughput sube hasta ≈900 resp/s y se aplana desde 400 clientes; desde ahí solo sube la latencia. El reparto se balancea solo: Python atiende menos cuando hay mucha carga.

14. Demo. Ver la lista de la demo más abajo.

15. Conclusiones. Leer los puntos de la diapositiva.

Lista para la demo en vivo

1. En la máquina servidor: `scripts/servidor.sh` (o `servidor.bat`). Anotar su IP.
2. En cada máquina de cómputo: `scripts/nodo_java.sh <ip>`, `scripts/nodo_python.sh <ip>`, `scripts/nodo_go.sh <ip>`.
3. `java -cp out cubes.client.AdminClient --host <ip> estado`: deben aparecer todos los nodos.
4. `java -cp out cubes.client.AdminClient --host <ip> crosscheck`: todos deben decir OK.
5. Abrir dos chats: `scripts/cliente.sh <ip>` con usuarios distintos; "Identificar" con una foto y con "Imagen de prueba".
6. Entrenamiento distribuido corto en vivo: `java -cp out cubes.client.AdminClient --host <ip> entrenar --images 1024 --langs all`.
7. Tolerancia a fallos: mientras entrena, cerrar un nodo con Ctrl+C y mostrar que su parte se reasigna.
8. Carga: `scripts/carga.sh <ip> demo 10,100,300 10` y comentar latencia y respuestas por segundo.

Antes de la exposición: abrir los puertos 5000 y 6000 en el firewall de la máquina servidor, probar que todas las máquinas se ven (`ping`), y tener `data/model.bin` en el servidor (si no, el chat responde con pesos aleatorios).

Preguntas probables y respuestas cortas

¿Qué es un micro chunk en su implementación? Un rango de trabajo independiente dentro de una capa: filtros en la convolución, neuronas en el MLP, canales en el pooling. El ejecutor reparte los rangos entre H hilos y espera a todos.

¿Por qué en Python usan procesos y no hilos? Por el GIL: en CPython solo un hilo ejecuta bytecode a la vez, así que los hilos no aceleran cálculo puro. Con `multiprocessing` cada proceso tiene su propio intérprete.

¿Por qué Python es mucho más lento? Es interpretado y no podemos usar numpy. Por eso el servidor mide la velocidad de cada nodo y le asigna menos imágenes a Python.

¿Cómo se sincronizan los nodos en el entrenamiento? El servidor es un servidor de parámetros síncrono: envía pesos y una parte del lote a cada nodo, espera todas las sumas de gradientes, actualiza y recién entonces manda el siguiente lote.

¿Qué pasa si se cae un nodo? El servidor lo detecta por el socket cerrado o por 10 s sin heartbeat. Las peticiones pendientes de ese nodo fallan y se reenvían a otro nodo vivo. El nodo, al volver, se registra de nuevo solo.

¿Por qué no usar paralelismo de modelo (capas en distintas máquinas)? Habría que enviar activaciones por la red en cada capa y para cada imagen. Con paralelismo de datos solo viajan pesos y gradientes una vez por lote (≈1 MB en cada sentido).

¿Por qué el distribuido no escala linealmente? Por la comunicación (pesos y gradientes por lote), la sincronización por lote (se espera al nodo más lento) y la parte secuencial del servidor (suma y actualización). Es la ley de Amdahl. Con lotes más grandes o red más rápida mejora.

¿Cómo garantizan que Java, Python y Go calculan lo mismo? Con la especificación común (`docs/CNN_SPEC.md`) y la prueba cruzada del servidor, que compara gradientes y probabilidades de cada nodo con Java.

¿Cómo se emparejan respuestas con peticiones? Cada trama lleva un id de petición. El servidor guarda un futuro por id y el hilo lector del nodo completa el futuro cuando llega la respuesta con ese id.

¿Qué pasa si llegan más consultas de las que se pueden atender? Se encolan (cola acotada de 20 000). La latencia sube mientras la cola crece; si se llena, se responde "servidor saturado" en vez de caerse.

¿Dónde está el backend? Dentro del proceso del servidor central, como módulo aparte (`Backend.java`): sesiones, idClientChat, historial y estadísticas.

¿Por qué big-endian? Es el orden de red estándar y el que usan por defecto `DataOutputStream` de Java, `struct` con `>` en Python y `binary.BigEndian` en Go.